

UNIT-2 (6-12 LAB GENAI)

Name: K. L. Khaviya varshini

Reg. No.: 192472381

PROGRAM6:

```
import os

from google import genai

client=genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

prompt=input("Enter your prompt: ").strip()

if not prompt:

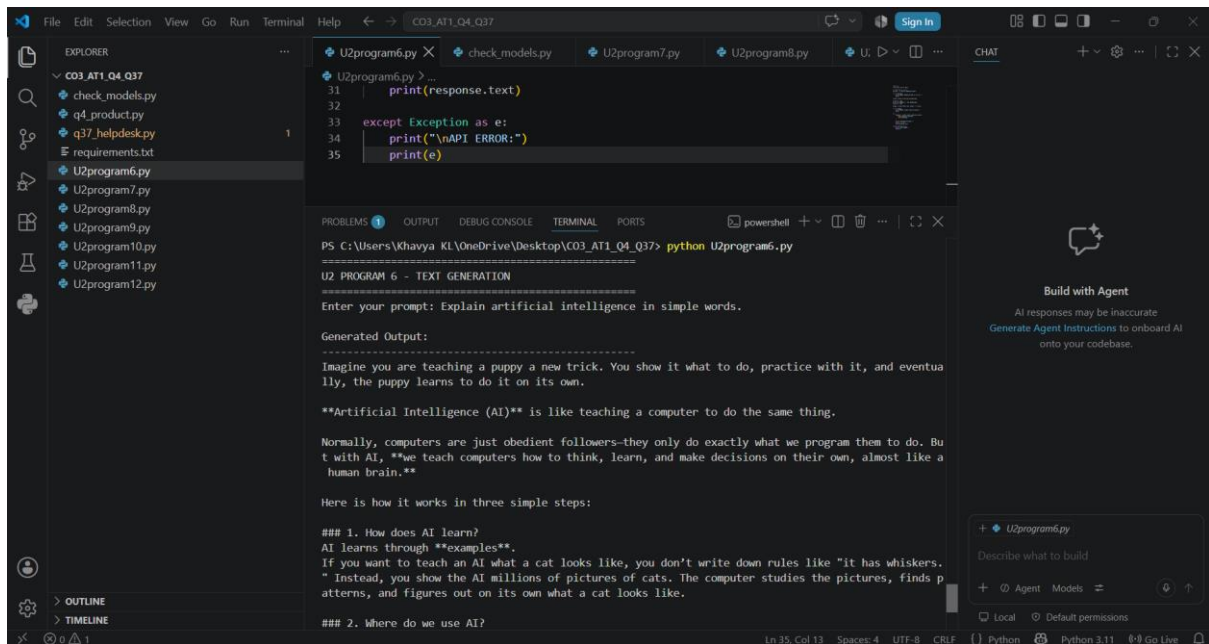
    print("Error: Prompt cannot be empty.")

else:

    response=client.models.generate_content(model="gemini-3.5-flash",contents=prompt)

    print("\nGenerated Output:")

    print(response.text)
```



The screenshot displays a Visual Studio Code (VS Code) interface with a Python script named `U2program6.py` open in the editor. The script uses the `google` library to interact with the Gemini API. The user has entered the prompt "Explain artificial intelligence in simple words." in the terminal. The output shows the generated response from the Gemini-3.5-flash model, which explains AI in simple terms, comparing it to teaching a puppy and a computer, and lists three simple steps for how AI works.

```
U2program6.py > ...
31 | print(response.text)
32 |
33 | except Exception as e:
34 |     print("\nAPI ERROR:")
35 |     print(e)
```

PS C:\Users\Khaviya KL\OneDrive\Desktop\CO3_AT1_Q4_Q37> python U2program6.py

U2 PROGRAM 6 - TEXT GENERATION

Enter your prompt: Explain artificial intelligence in simple words.

Generated Output:

Imagine you are teaching a puppy a new trick. You show it what to do, practice with it, and eventually, the puppy learns to do it on its own.

****Artificial Intelligence (AI)**** is like teaching a computer to do the same thing.

Normally, computers are just obedient followers—they only do exactly what we program them to do. But with AI, ****we** teach computers how to think, learn, and make decisions on their own, almost like a human brain.

Here is how it works in three simple steps:

1. How does AI learn?

AI learns through ****examples****.

If you want to teach an AI what a cat looks like, you don't write down rules like "it has whiskers." Instead, you show the AI millions of pictures of cats. The computer studies the pictures, finds patterns, and figures out on its own what a cat looks like.

2. Where do we use AI?

PROGRAM7:

```
import os

from google import genai

client=genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

for i in range(3):

    prompt=input(f'Enter Prompt {i+1}: ').strip()

    if not prompt:

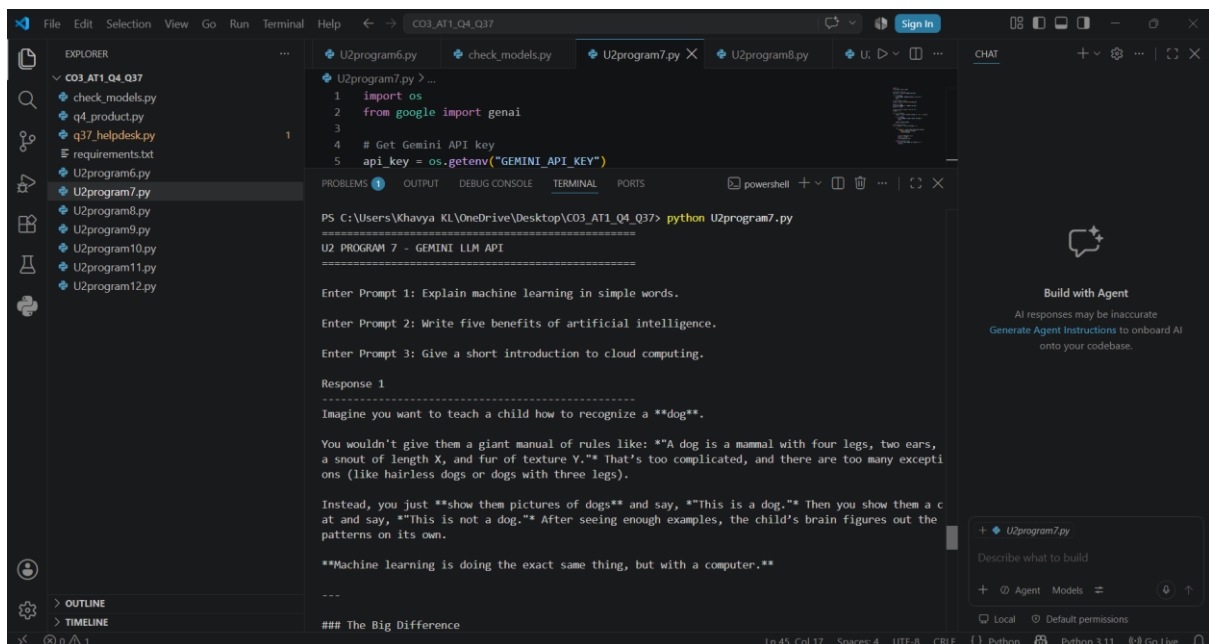
        print("Error: Prompt cannot be empty.")

        break

    response=client.models.generate_content(model="gemini-3.5-flash",contents=prompt)

    print(f'\nResponse {i+1}:')

    print(response.text)
```



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files in a directory named 'CO3_AT1_Q4_Q37', including 'U2program7.py'. The main editor window displays the Python code from the previous block. The integrated terminal at the bottom shows the command 'python U2program7.py' being executed. The output of the program is displayed in the terminal, showing three prompts entered by the user and the corresponding responses generated by the Gemini 3.5 Flash model. The first response is about teaching a child to recognize a dog using machine learning. The second response is about the benefits of artificial intelligence. The third response is about cloud computing. The terminal also shows the prompt 'Enter Prompt 3: Give a short introduction to cloud computing.' and the response 'Response 1: Imagine you want to teach a child how to recognize a dog. You wouldn't give them a giant manual of rules like: "A dog is a mammal with four legs, two ears, a snout of length X, and fur of texture Y." That's too complicated, and there are too many exceptions (like hairless dogs or dogs with three legs). Instead, you just show them pictures of dogs and say, "This is a dog." Then you show them a cat and say, "This is not a dog." After seeing enough examples, the child's brain figures out the patterns on its own. Machine learning is doing the exact same thing, but with a computer.'

PROGRAM8:

```
import os

from huggingface_hub import InferenceClient

token=os.getenv("HF_TOKEN")

if not token:

    print("Error: HF_TOKEN is not set.")

else:

    prompt=input("Enter your prompt: ").strip()

    if not prompt:

        print("Error: Prompt cannot be empty.")

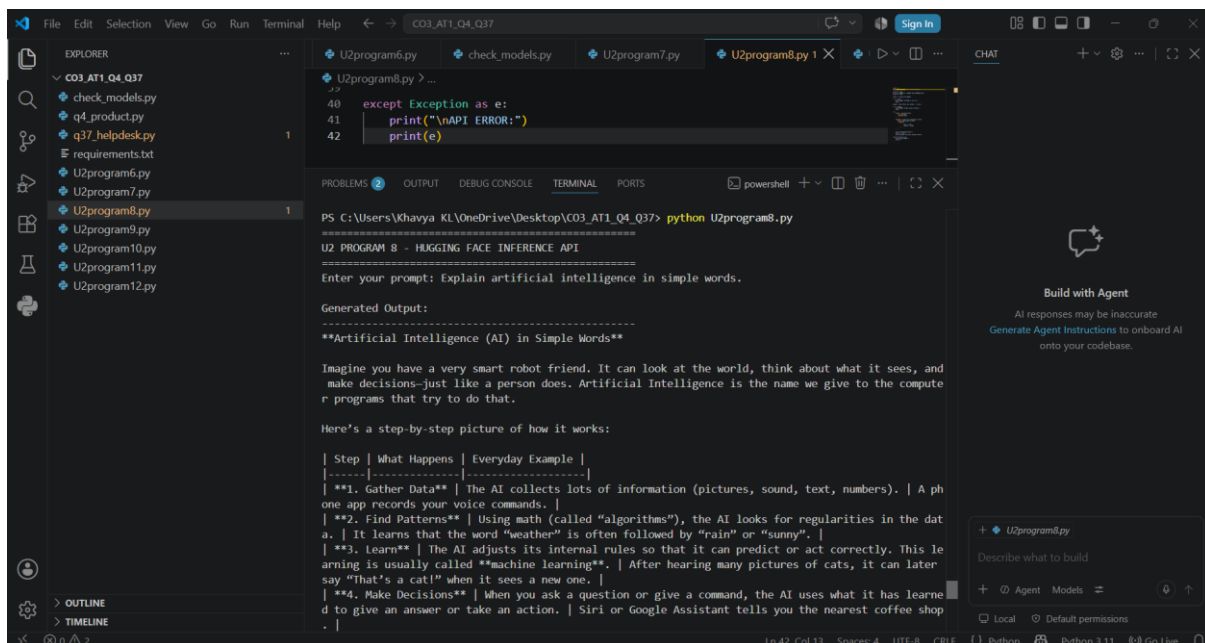
    else:

        client=InferenceClient(api_key=token,provider="auto")

        response=client.chat.completions.create(model="openai/gpt-oss-120b",messages=[{"role":"user","content":prompt}])

        print("\nGenerated Output:")

        print(response.choices[0].message.content)
```



The screenshot displays a Visual Studio Code (VS Code) interface with a Python script named `U2program8.py` open in the editor. The script is designed to interact with the Hugging Face Inference API. It prompts the user to enter a prompt, and if a valid prompt is provided, it uses the `InferenceClient` to generate a response from the `openai/gpt-oss-120b` model. The output of the script is displayed in the terminal window, showing the generated text for the prompt "Explain artificial intelligence in simple words." The output is a detailed explanation of AI, including its definition, how it works, and examples of its applications.

```
PS C:\Users\Khavya KL\OneDrive\Desktop\CO3_AT1_Q4_Q37> python U2program8.py
U2 PROGRAM 8 - HUGGING FACE INFERENCE API
Enter your prompt: Explain artificial intelligence in simple words.

Generated Output:
=====
**Artificial Intelligence (AI) in Simple Words**

Imagine you have a very smart robot friend. It can look at the world, think about what it sees, and
make decisions-just like a person does. Artificial Intelligence is the name we give to the compute
r programs that try to do that.

Here's a step-by-step picture of how it works:

| Step | What Happens | Everyday Example |
|-----|-----|-----|
| **1. Gather Data** | The AI collects lots of information (pictures, sound, text, numbers). | A ph
one app records your voice commands. |
| **2. Find Patterns** | Using math (called "algorithms"), the AI looks for regularities in the dat
a. | It learns that the word "weather" is often followed by "rain" or "sunny". |
| **3. Learn** | The AI adjusts its internal rules so that it can predict or act correctly. This le
arning is usually called "machine learning". | After hearing many pictures of cats, it can later
say "That's a cat!" when it sees a new one. |
| **4. Make Decisions** | When you ask a question or give a command, the AI uses what it has learne
d to give an answer or take an action. | Siri or Google Assistant tells you the nearest coffee shop
. |
```

PROGRAM9:

```
import os

from google import genai

client=genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

problem=input("Enter the programming problem: ").strip()

if not problem:

    print("Error: Problem cannot be empty.")

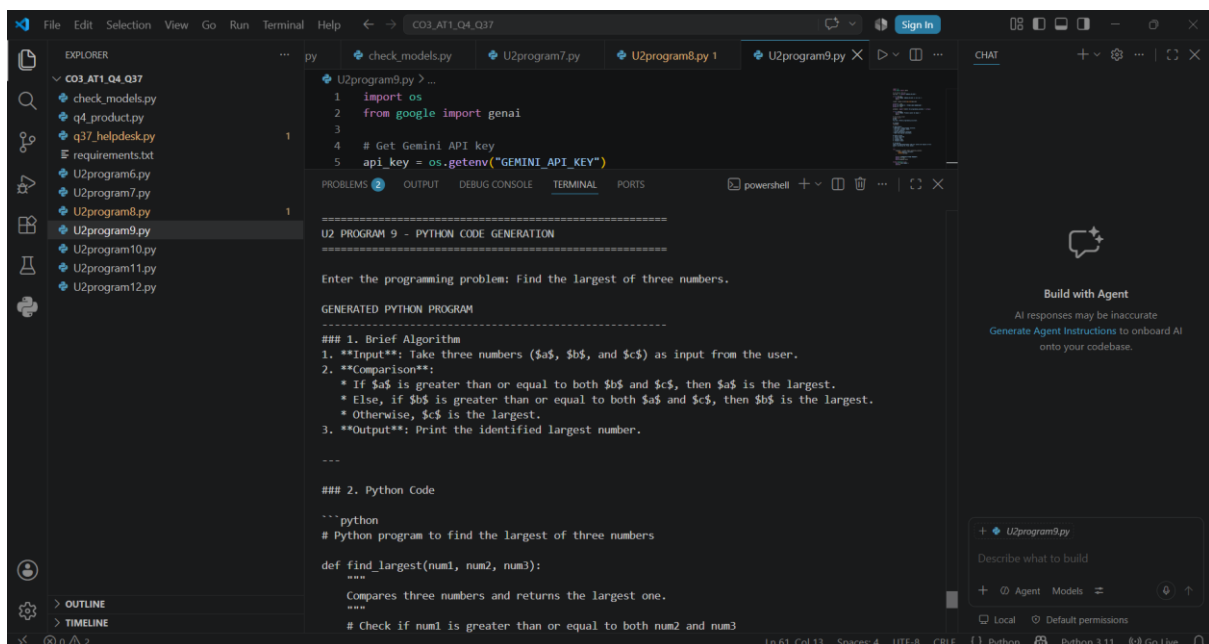
else:

    prompt=f"""Generate a Python program for the following problem.
Problem: {problem}
Requirements:
- Use clear variable names.
- Include comments.
- Make the program executable.
- Provide an algorithm, Python code, example input and example output.
- Check the code for syntax and logical errors."""

    response=client.models.generate_content(model="gemini-3.5-flash",contents=prompt)

    print("\nGenerated Python Program:")

    print(response.text)
```



PROGRAM10:

```
import os

from google import genai

client=genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

schema="""Database: CollegeDB

Table: Student

Columns: student_id INT, name VARCHAR(50), department VARCHAR(50), marks INT"""

requirement=input("Enter the SQL requirement: ").strip()

if not requirement:

    print("Error: SQL requirement cannot be empty.")

else:

    prompt=f"""Generate an SQL query using the following database schema.

Schema:

{schema}

Requirement:

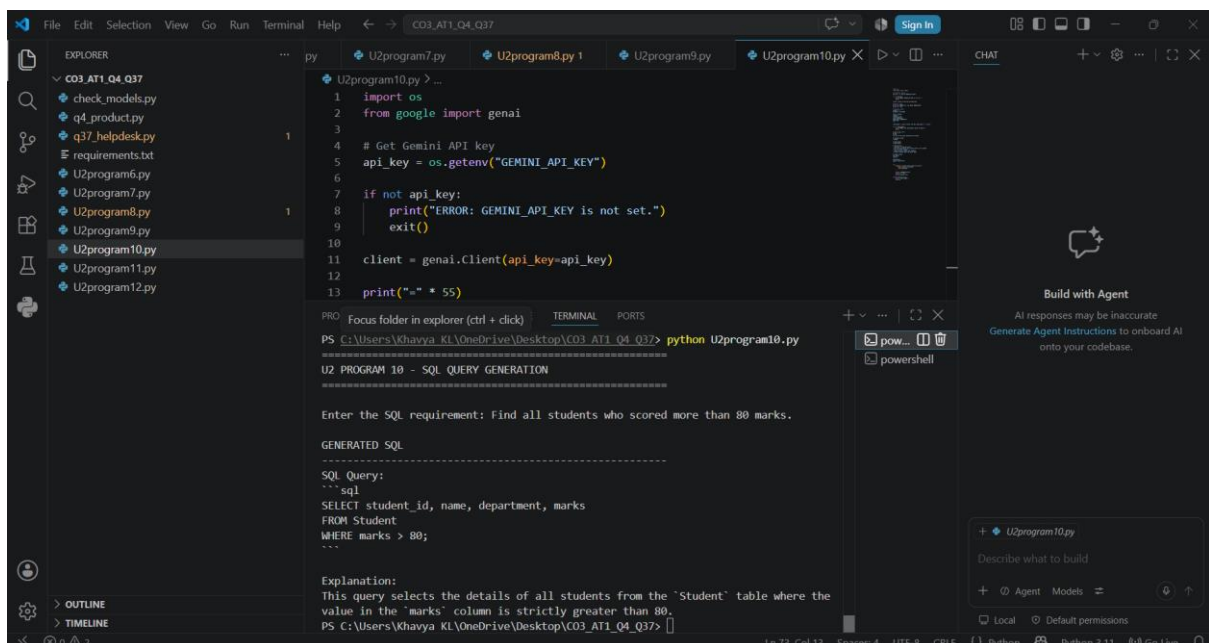
{requirement}

Generate a valid SQL query using only the given table and columns and briefly explain it."""

    response=client.models.generate_content(model="gemini-3.5-flash",contents=prompt)

    print("\nGenerated SQL:")

    print(response.text)
```



The screenshot shows a Visual Studio Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays the Python script for Program 10. The terminal shows the command to run the script and its output, which includes the generated SQL query and its explanation.

```
File Edit Selection View Go Run Terminal Help CO3_AT1_Q4_Q37
EXPLORER
  CO3_AT1_Q4_Q37
    check_models.py
    q4_product.py
    q37_helpdesk.py
    requirements.txt
    U2program6.py
    U2program7.py
    U2program8.py
    U2program9.py
    U2program10.py
    U2program11.py
    U2program12.py
  U2program10.py
  U2program11.py
  U2program12.py

U2program10.py
1 import os
2 from google import genai
3
4 # Get Gemini API key
5 api_key = os.getenv("GEMINI_API_KEY")
6
7 if not api_key:
8     print("ERROR: GEMINI_API_KEY is not set.")
9     exit()
10
11 client = genai.Client(api_key=api_key)
12
13 print("-" * 55)

PRO Focus folder in explorer (ctrl + click) TERMINAL PORTS
PS C:\Users\Khavya KL\OneDrive\Desktop\CO3_AT1_Q4_Q37> python U2program10.py
U2 PROGRAM 10 - SQL QUERY GENERATION

Enter the SQL requirement: Find all students who scored more than 80 marks.

GENERATED SQL
-----
SQL Query:
'''sql
SELECT student_id, name, department, marks
FROM Student
WHERE marks > 80;
'''

Explanation:
This query selects the details of all students from the 'Student' table where the
value in the 'marks' column is strictly greater than 80.
PS C:\Users\Khavya KL\OneDrive\Desktop\CO3_AT1_Q4_Q37>
```

```
import os

from google import genai

client=genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

statement=input("Enter a statement: ").strip()

if not statement:
```

PROGRAM12:

```
import os

from google import genai

client=genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

text=input("Enter text to summarize: ").strip()

if not text:

    print("Error: Text cannot be empty.")

else:

    prompts=[

        f"Summarize the following text:\n{text}",

        f"Summarize the following text in 3 sentences and include only the important information:\n{text}",

        f"Summarize the following text in exactly 3 sentences. Include the main points, use clear language, avoid repetition, and do not add information not present in the text.\n{text}"

    ]

    for i,prompt in enumerate(prompts,1):

        response=client.models.generate_content(model="gemini-3.5-flash",contents=prompt)

        print(f"\nPrompt {i}:")

        print(response.text)

    refined=f"Create a clear 3-sentence summary. Include the main idea and important points, use simple language, avoid repetition, and do not add outside information.\nText:\n{text}"

    response=client.models.generate_content(model="gemini-3.5-flash",contents=refined)

    print("\nRefined Prompt Output:")

    print(response.text)
```

